DKD

Backend Interview Guide

Easy, current, and honest for SDE interviews

What this guide is: a short explanation of the backend you actually built.
How to describe the UI: “an abstract REST client” - this can be Swagger or any future frontend.
What not to claim: frontend engineering, training an AI model, or guaranteed Kafka/email delivery.

Memory rule

Do not memorize every class. Tell the request story: **client -> security -> controller -> service -> database/cache/external API -> DTO response**.

Made for Tejas

1. DKD in one clear story

DKD (Dimaag Ka Darpan) is a Spring Boot backend for a personal reflection product. An authenticated client can create and manage journal entries, search them, see dashboard and weekly analytics, receive reflection prompts, optionally request AI insight for a journal, and receive a weekly reflection email.

> **45-second answer**
> “DKD is a Spring Boot backend for a personal reflection platform. It exposes REST APIs to an authenticated client for journaling, search, dashboard and weekly analytics. I use Spring Security with JWT, MongoDB for persistent data, Redis for selected cacheable values, an LLM API for opt-in journal reflections, and Kafka to decouple the weekly email workflow. The main design goal was privacy-aware reflection: user data stays scoped to the authenticated user, and AI requires explicit consent.”

Architecture in one picture

Abstract REST client (Swagger today; any UI later)

↓

Spring Security + JWT filter

↓

Controllers -> services -> response DTOs

↓

MongoDB: users, journals, prompts, AI insights, configuration

↓

Redis: weather and AI system-prompt cache

↓

Optional LLM API: one AI-enabled journal

↓

Sunday scheduler -> Kafka -> weekly email service -> SMTP

Use this wording for the client

“The backend is client-agnostic. I tested it using Swagger and REST calls. A frontend can consume the same documented endpoints, but my project contribution is the backend architecture and API contract.”

2. Authentication and authorization

Authentication answers *who is making this request?* Authorization answers *what are they allowed to do?* DKD uses Spring Security for both and JWT as the token format. Normal protected routes need authentication; admin and prompt-management routes need ADMIN.

Signup and login

POST /public/signup

↓

Password is hashed with BCrypt; USER role is assigned

↓

User is stored in MongoDB

↓

POST /public/login

↓

AuthenticationManager + UserDetailsService verify credentials

↓

DKD returns a signed, short-lived JWT

Protected request

Client sends Authorization: Bearer

↓

JwtFilter reads and validates signature and expiry

↓

UserDetailsService reloads current user and roles from MongoDB

↓

SecurityContextHolder stores the authenticated identity for this request

↓

Controller uses SecurityUtil.getCurrentUsername()

↓

Service returns only that user's data

Interview-safe line
“BCrypt hashes passwords; it does not encrypt them. A JWT is signed, so I do not put passwords or secrets inside it. Roles are reloaded from MongoDB during JWT processing.”

3. Journal API, ownership, and stable IDs

A journal entry is a MongoDB document. The User document keeps DBRef references to its own journal entries. This lets the service verify that an entry belongs to the authenticated user before reading, updating, deleting, or sending it to AI.

Core journal fields

- title and content
- date
- sentiment: HAPPY, SAD, ANGRY, LOW, NEUTRAL, or ANXIOUS
- tags and reflectionPrompt
- aiEnabled for per-journal AI consent

Important API contract

MongoDB stores IDs as ObjectId internally. REST responses use a small response DTO that converts the ID to a normal 24-character hexadecimal string. This keeps persistence details out of the client contract.

```
{ "id": "507f1f77bcf86cd799439011", "title": "A pause", "sentiment": "HAPPY", "aiEnabled": false
}
```

When POST /journal saves an entry, the service uses the exact object returned by MongoDB save, then returns 201 Created with that real ID. It does not query for the “latest” journal, which could be unsafe under concurrent requests.

Search and filters

JournalSearchService searches titles only, case-insensitively, supports an optional sentiment and inclusive start/end dates, and sorts by date descending by default. All filtering begins from the authenticated user's journal list, so one user cannot search another user's journals.

4. Dashboard and weekly analytics

The dashboard and weekly analytics are calculated from the current user's journal history. They are not stored as a separate dashboard collection, so the output stays derived from the user’s real entries.

Dashboard returns

- total entries
- current streak
- entries this week and this month
- dominant sentiment
- last entry date

Weekly analytics returns

- Monday-to-today date range
- total entries and sentiment breakdown
- dominant sentiment
- current streak reused from DashboardService
- average entries per elapsed day
- journal consistency = unique journaling days / elapsed days * 100

> **How to explain the streak**
> “I put streak logic in DashboardService and reused it from AnalyticsService instead of duplicating it. It counts consecutive journal dates, treats multiple entries on one day as one day, and allows a streak to remain active until a full day is missed.”

Sentiment, not mood

The product uses one structured field called **Sentiment**. If AI is not used for an entry, the client can send the user-selected sentiment. If AI is enabled and insight is generated, the validated AI-detected sentiment is saved on that same field. There is no separate mood field to keep in sync.

5. Reflection prompts and AI insight

Reflection prompts are independent data managed by an admin. The app returns a deterministic active prompt for the current day, so refreshes do not randomly change the prompt. Deleting a prompt is a soft delete: it is marked inactive instead of physically removed.

Two levels of AI consent

Account-level AI Reflection setting must be enabled

↓

This gives the user a global switch for AI features

↓

The individual journal must also have aiEnabled = true

↓

Only then can POST /ai/generate/{journalId} call the LLM

AI request workflow

Verify authenticated user owns this journal

↓

Verify global and per-journal AI consent

↓

Return existing insight if it was already generated

↓

Build structured journal context

↓

Call LLM API with system instruction + journal prompt

↓

Validate JSON and Sentiment enum

↓

Save AIInsight; update journal sentiment; return DTO

The journal context can contain title, existing sentiment if present, tags, reflection prompt, and content. It does not send username, email, password, JWT, or roles to the AI provider.

> **Important honest line**
> “I integrated an LLM API; I did not train an AI model. The engineering work is consent checks, ownership checks, prompt construction, response validation, storage, and reuse.”

6. Configuration, Redis, and external calls

DKD separates persistent data from temporary cache. MongoDB is the source of truth. Redis is used only where a short-lived cached value saves repeat work.

AppCache vs Redis

AppCache	Redis
In-process Java map loaded from the MongoDB configuration collection.	Separate in-memory cache with time-to-live.
Used for application configuration such as model/base URL/system-prompt source.	Used for weather results and a cached AI system prompt.

Cache-aside in DKD

Need weather or AI system prompt

↓

Check Redis

↓

Cache hit: return value

↓

Cache miss: load original source/AppCache

↓

Store with TTL and return value

If Redis is unavailable, the code treats it like a cache miss and falls back to the source. The system must not lose journals or AI insights because those are stored in MongoDB.

7. Weekly email and Kafka

Every Sunday at 9 AM, the scheduler finds users eligible for the weekly report. It publishes a small event containing the username. A Kafka consumer later builds and sends the report.

Spring Scheduler

↓

WeeklyReflectionProducer

↓

Kafka topic: weekly-reflection-topic

↓

WeeklyReflectionConsumer

↓

WeeklyReflectionEmailService

↓

SMTP email

The report uses weekly analytics plus stored AI insights. It does **not** call the LLM again in the scheduler. It combines already saved summaries and reuses the latest positive observation, reflection question, and encouragement. If no AI insight exists, it uses helpful default text while normal analytics still work.

Why Kafka?

“Kafka decouples the Sunday trigger from the heavier report and email processing. The scheduler publishes an event; the consumer handles the work independently. For this project, a direct call would be simpler, but Kafka let me practice an event-driven producer-consumer flow.”

Current limitation to state honestly

There is a direct-email fallback only when the immediate producer call throws. Kafka send itself is asynchronous, so this is not a full delivery guarantee. Production hardening would add delivery callbacks, retries, dead-letter handling, idempotency, and monitoring.

8. MongoDB, privacy, and scalability

Why MongoDB?

“Journal data is naturally document-shaped and my schema evolved with fields such as tags, reflection prompts, sentiment, AI consent, and insights. MongoDB let me model that flexibly. I would not claim it is always better than SQL: a product with many relational reports and joins could benefit from a relational database.”

How does DKD protect privacy?

“Every protected request is authenticated by JWT. Services derive the current username from Spring Security and check journal ownership before returning, changing, or sending journal content to AI. Passwords are BCrypt hashes. AI is opt-in at both account and entry level, and the LLM prompt excludes identity and security data.”

Can the application owner read journal content?

“The backend needs access to store and serve a user's content, so it is not end-to-end encrypted. Access is protected by authentication and authorization, but an operator with production database access could potentially access data. A stronger privacy roadmap would include least-privilege production access, audit logs, encryption at rest, secret management, and possibly client-side encryption with clear trade-offs.”

How would it scale?

“First I would measure. I would add database indexes for frequent lookups such as username and consider moving search/filter work to database-side queries with pagination. For larger workloads, I would scale stateless application instances behind a load balancer, keep MongoDB and Redis managed/scalable, and harden Kafka consumer/retry behavior. I would not prematurely add microservices.”

9. Testing, API quality, and deployment

The project uses JUnit for test execution and assertions, and Mockito to replace dependencies such as repositories, Redis, Kafka, SMTP, and the LLM API during isolated service tests.

What to say about testing

“I wrote focused unit tests around business rules: dashboard and weekly calculations, search/filter behavior, AI consent and ownership checks, response DTO mapping, reflection prompts, Kafka producer calls, and weekly-report composition. Unit tests are not the same as a full production integration test; I would extend integration tests for MongoDB, Redis, Kafka, SMTP, and the external AI provider.”

API contract quality

- Controllers are thin: authentication context, service call, DTO response.
- Services hold business rules and reuse shared calculations where possible.
- Response DTOs avoid leaking persistence-specific ObjectId internals.
- Journal IDs are normal 24-character hex strings in REST responses.
- Swagger/OpenAPI documents endpoints and lets a client test them.

Deployment discussion

Docker packages the Spring Boot application with its runtime. Environment-specific configuration and secrets should be supplied outside source control. Production deployment still needs health checks, logs/metrics, HTTPS, managed MongoDB/Redis/Kafka where appropriate, and a reliable email provider.

> **Do not overclaim**
> Say “I built and tested the backend flow” rather than “it is production-ready.” Strong interviews reward accurate trade-offs.

10. High-probability interview questions

What happens when a user creates a journal?

The request passes the JWT filter. The controller gets the authenticated username through SecurityUtil, the service saves the journal as a MongoDB document, adds its DBRef to that user's journal list in a transaction, and returns a response DTO with the saved ObjectId converted to a hex string.

Why does the client need the journal ID?

The ID is a stable identifier for later GET, update, delete, and AI-insight requests. The backend maps the Mongo ObjectId to a 24-character hex string in the response, then converts the path string back to ObjectId internally.

JWT vs OAuth?

JWT is a signed token format. OAuth 2.0 is an authorization framework for delegated access, often used with external login providers. DKD issues JWTs for its own users; OAuth would be useful for an external provider such as Google login.

Why store an AI insight?

It avoids repeatedly sending the same journal to the LLM, reduces cost and latency, and lets the weekly report reuse insight already generated with consent.

Redis vs MongoDB?

MongoDB is persistent source-of-truth data. Redis is a fast temporary cache. Clearing Redis should not lose journals, users, or AI insights.

What would you improve next?

Database-side search with pagination, stronger request validation and consistent error responses, more real integration tests, observability, production secret management, and reliable Kafka retry/dead-letter/idempotency design.

11. Last 10-minute revision

Remember the four stories, not every implementation detail.

1. Identity

Signup -> BCrypt hash -> MongoDB. Login -> AuthenticationManager -> JWT. Protected request -> JwtFilter -> SecurityContextHolder -> current user.

2. Journal

Authenticated user -> ownership check -> journal CRUD/search -> MongoDB. REST DTO turns ObjectId into a normal hex string.

3. Insight

Global AI switch + per-journal switch -> ownership -> existing insight check -> structured prompt -> LLM -> validate sentiment -> save and reuse.

4. Weekly report

Sunday scheduler -> Kafka event -> consumer -> analytics + stored insights -> SMTP email. No new LLM call in scheduler.

30-second final answer
“DKD is a Spring Boot backend for a personal reflection platform. It offers secure journal APIs, search, dashboard and weekly analytics through a client-agnostic REST contract. I use JWT and Spring Security for user-scoped access, MongoDB for persistent documents, Redis for selected caching, optional LLM integration for individual journal reflections, and Kafka for the weekly email workflow. The most important design choices were consent, ownership checks, and keeping API responses clean through DTOs.”

If you blank out: tell the request story first. The technical names will come back.